

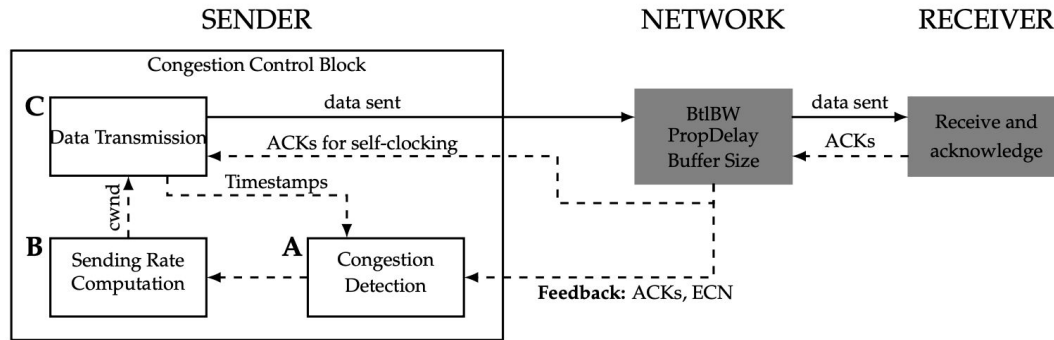
Orca 2.0!

Classic Meets *Even More* Modern:
a (Slightly) Pragmatic Learning-Based
Congestion Control for the Internet

Krithi Sub, Mircea Tatulescu

TCP Congestion Control

Goal: prevent network overload by regulating the data transmission between a sender and receiver.



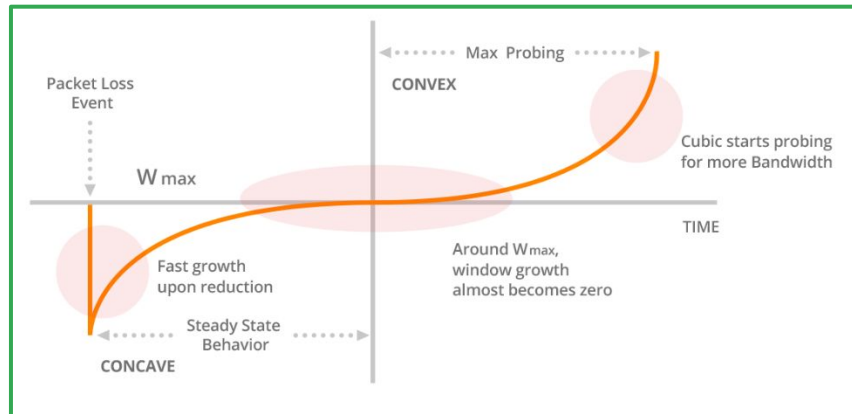
A block diagram for end-to-end TCP congestion control.

Can such an algorithm
be fast and stable?

TCP Cubic

Pros:

- Stable on wired networks.
- Low CPU overhead.
- Aggressive congestion window growth.



Intended behavior of Cubic.

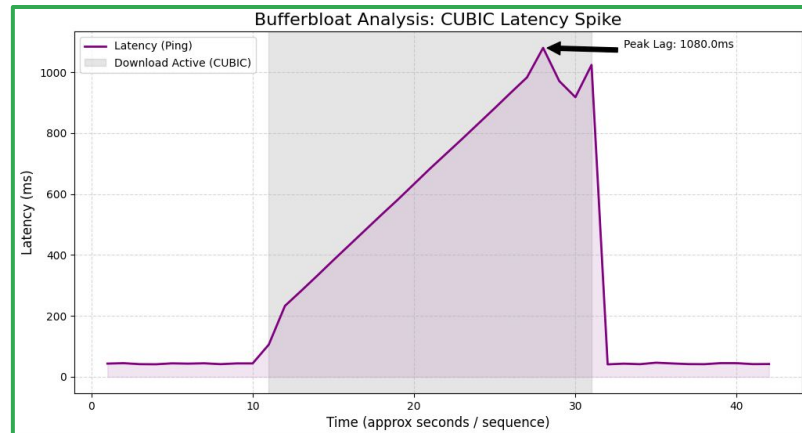
TCP Cubic

Pros:

- Stable on wired networks.
- Low CPU overhead.
- Aggressive congestion window growth.

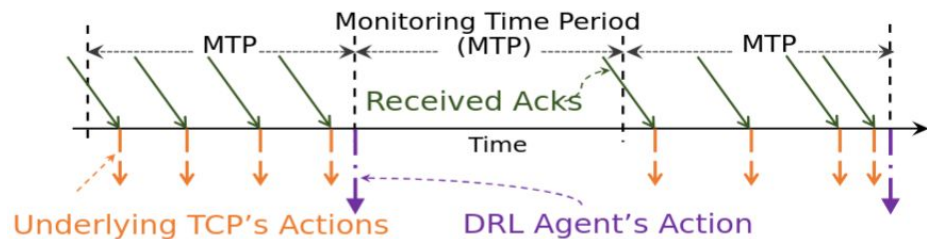
Cons:

- Causes instant queue bloating.
- Unstable on variable networks.
- Reactive not predictive.

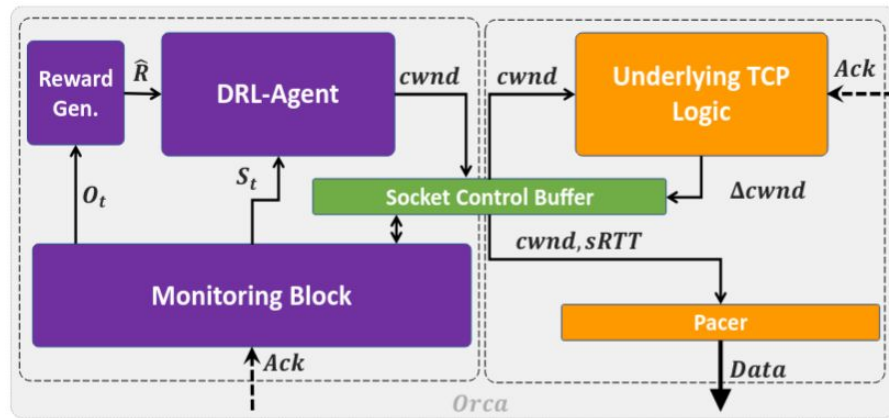


Bufferbloat on Cubic.
[iperf3]

Orca



Simplified time-diagram of two-level control logic.



High-level block diagram of Orca.

Don't replace Cubic → *control* it.

Table 1: Statistics generated by Monitoring block

thr	The average delivery rate (throughput)
l	The average loss rate of packets
d	The average delay of packets
n	The number of valid acknowledgment packets
m	The time between the last report and current report
$sRTT$	The smooth RTT of packets so far
$cwnd_u$	The current congestion window
thr_{max}	The maximum value of delivery rate so far
d_{min}	The minimum value of packet delay so far

Orca 2.0!

Goal 1:

Port the entire Orca kernel from Linux 4.13 to Linux 6.16.

Goal 2:

Improve DRL agent and scale decision frequency based on round-trip time.

Goal 3:

Autonomously fallback to an optimal congestion control algorithm based on current network conditions.

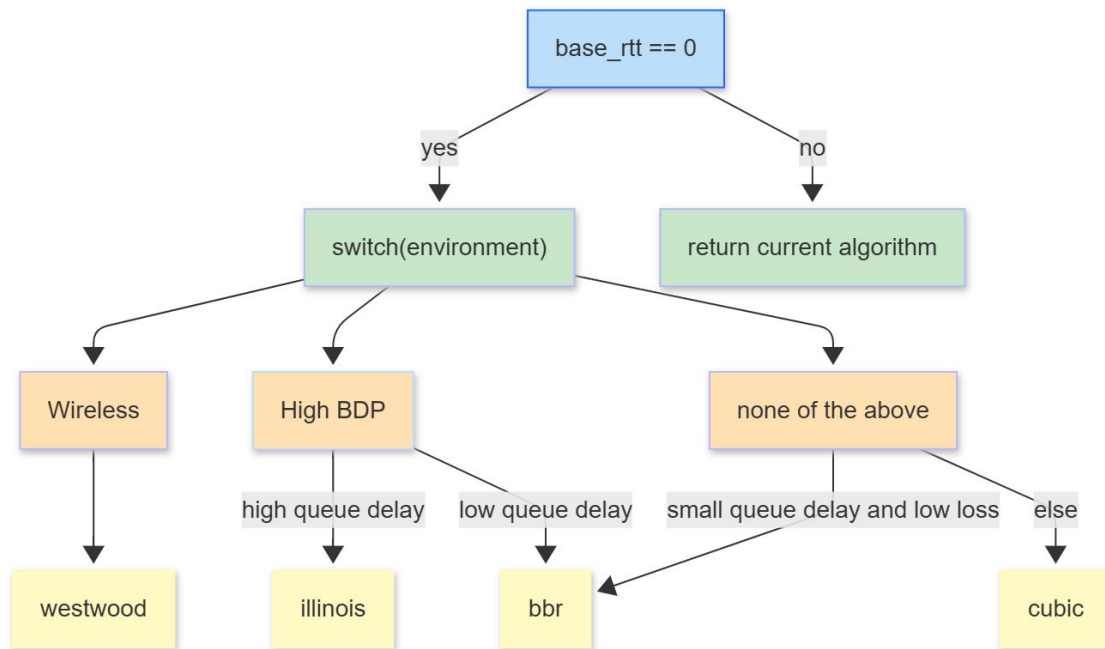
Kernelspace Modifications

1. Porting Original Patch

- `.../ipv4/tcp_deepcc.c`: monitoring and action enforcer blocks.
- `.../tcp_input.c`: input hooks, `tcp_clean_rtx_queue` and `tcp_cong_control`
- `.../tcp.h`: struct for `{min_urtt, avg_urtt, avg_thr, pre_lost}`
- `.../ipv4/tcp.c`: new socket options and handlers `{TCP_CWND, TCP_CWND_CAP, TCP_CWND_MIN, TCP_DEEPCC_ENABLE}`
- `.../ipv4/tcp_cubic.c`: function to override `cwnd`
`natcp_update_by_app`

Kernelspace Modifications

2. Implementing Algorithm Switching

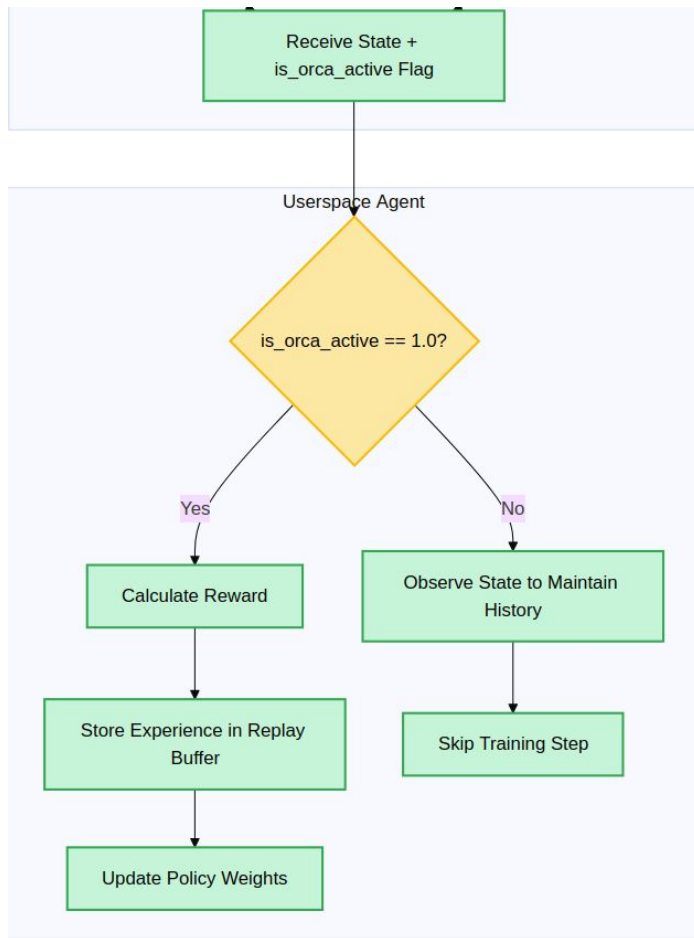
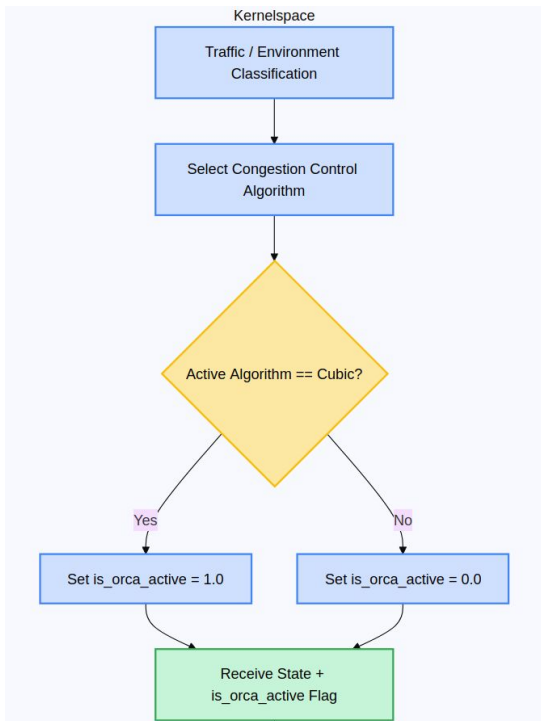


Existing Algorithms (more exist)

Algorithm	Ideal Environment	Main Parameters
DCTCP	Datacenter networks with Explicit Congestion Notifications (ECN), low round trip time	ECN marking ratio RTT/ACK rate
Westwood	Wireless paths with variable capacity	ACK rate
Veno	Wireless networks with congestion	Base RTT vs. current RTT (queuing delay)
BBR	High Bandwidth-Delay product paths	Bottleneck bandwidth from delivery rate Minimum RTT
Illinois	High-speed, high-BDP paths with significant queueing delay	Packet loss Queueing delay
Cubic	Generic Internet; without ECN	Packet loss

Userspace Modifications

1. Supporting Algorithm Switching



Userspace Modifications

2. Monitoring Time Period

Feedback Loop(RTT) >> Control Loop(MTP)

$RTT = 100\text{ms}$, $MTP = 20\text{ms}$ (fixed)

→ 5 blind actions



$MTP_{next} = \text{CLAMP}(RTT_{current}, 20\text{ms}, 100\text{ms})$

→ $MTP = RTT$

Agent makes five decisions before
seeing the result of the first one.

[Action] wait [Feedback] →
[Action]

Userspace Modifications

3. Reward Function

$$R = (\alpha \cdot \text{Throughput} - \beta \cdot \text{Loss} \times \text{DelayFactor})$$



$$R = (\alpha \cdot \log(\text{Throughput}) - \beta \cdot \text{Delay} - \gamma \cdot \text{Loss})$$

tuned values:

$$\alpha = 20.0$$

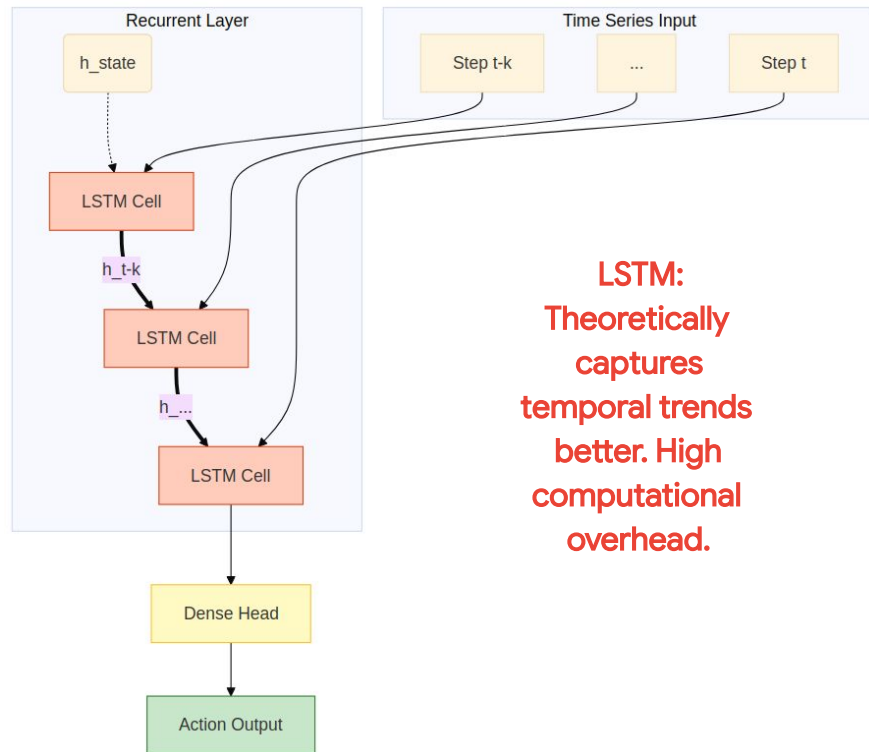
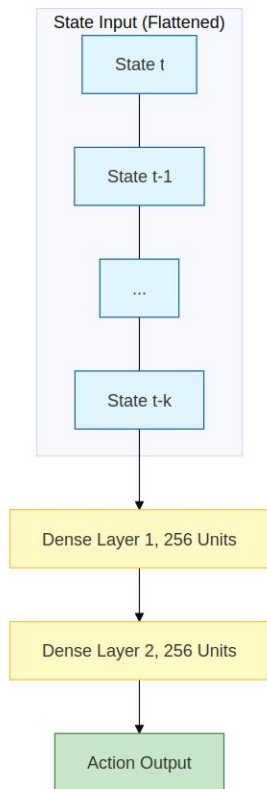
$$\beta = 50.0$$

$$\gamma = 200.0$$

Userspace Modifications

4. Architecture

Multilayer Perceptron:
timesteps all concatenated into a single feature vector.



LSTM:
Theoretically captures temporal trends better. High computational overhead.

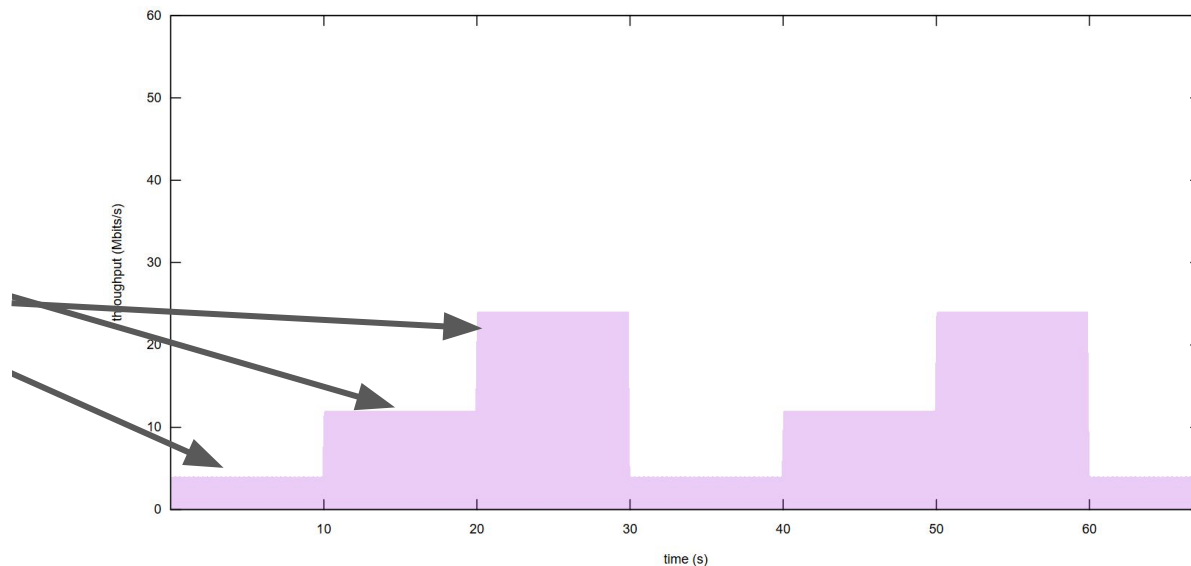
Implementations

1. Baseline Cubic
2. Baseline Orca
3. Orca + Log Reward
4. LSTM + Log Reward
5. Orca + All Modifications

Experiments

Mahimahi Traces

20206	23437
20207	23437
20208	23438
20209	23438
20210	23439
20211	23439
20212	23440
20213	23440
20214	23441
20215	23441
20216	23442
20217	23442



Experiments

`Queue Size  $\geq$  Bandwidth  $\times$  Round-Trip Time`

Small Queues:

- Low latency.
- Fast congestion signaling.

Large Queues:

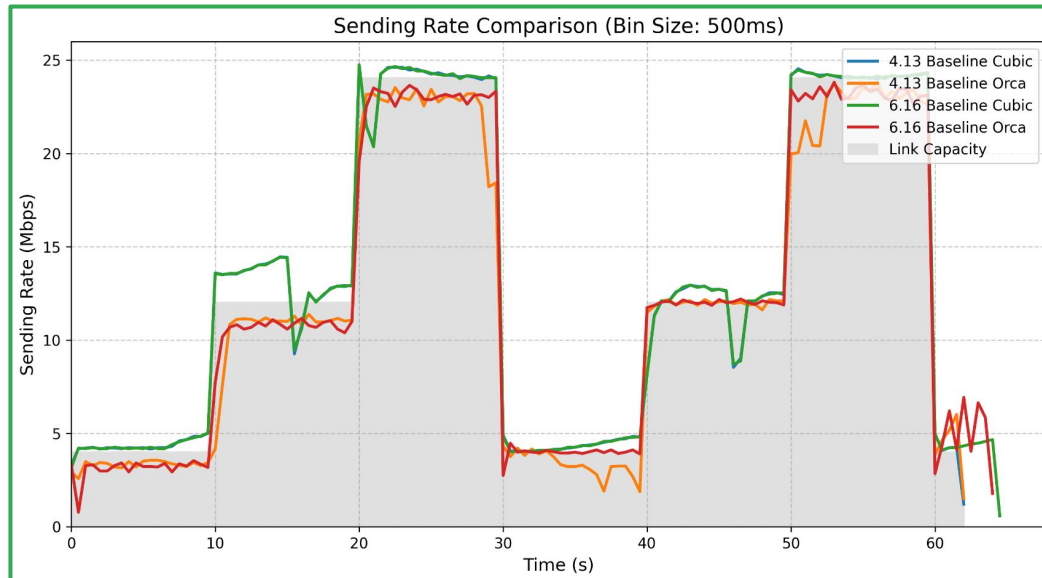
- High throughput.
- Absorbs bursts.

Problems?

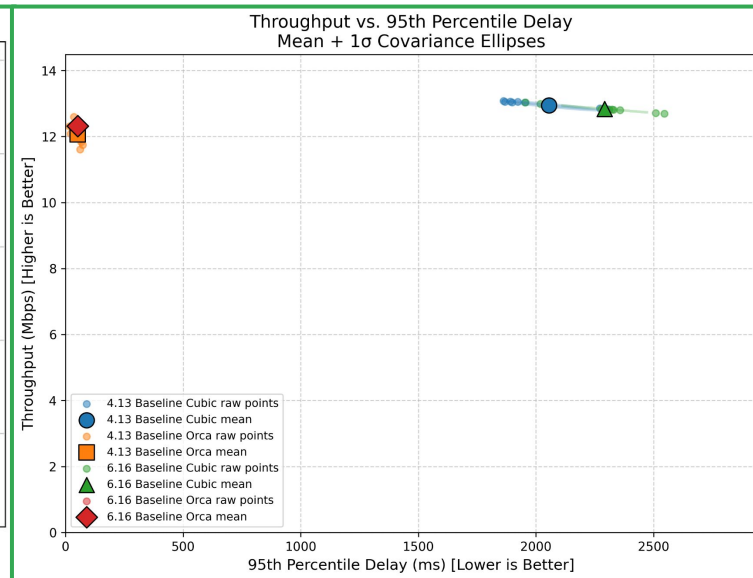
Bursts can lead to packet drops.

Underutilization of link due to slow scaling of congestion control algorithms.

Kernel Comparisons



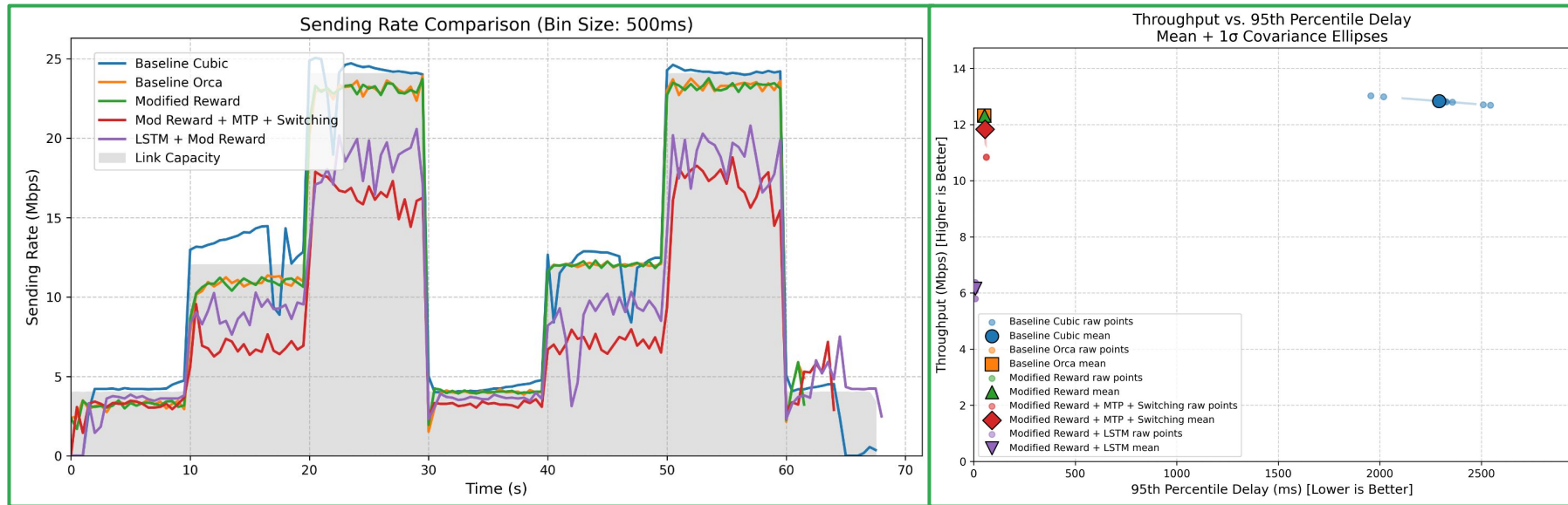
Cubic exceeds link capacity, Orca does not.



Cubic is unstable and has high delay.

Step Trace.

Implementation Comparisons

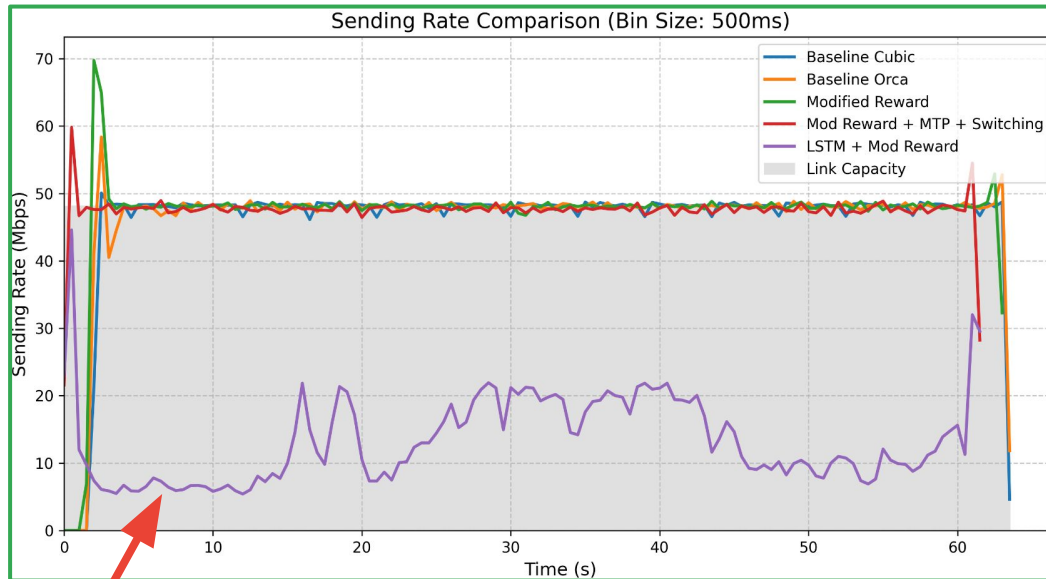


Most modifications oscillate and perform poorly.

Comparatively, Cubic has extreme delay variance.

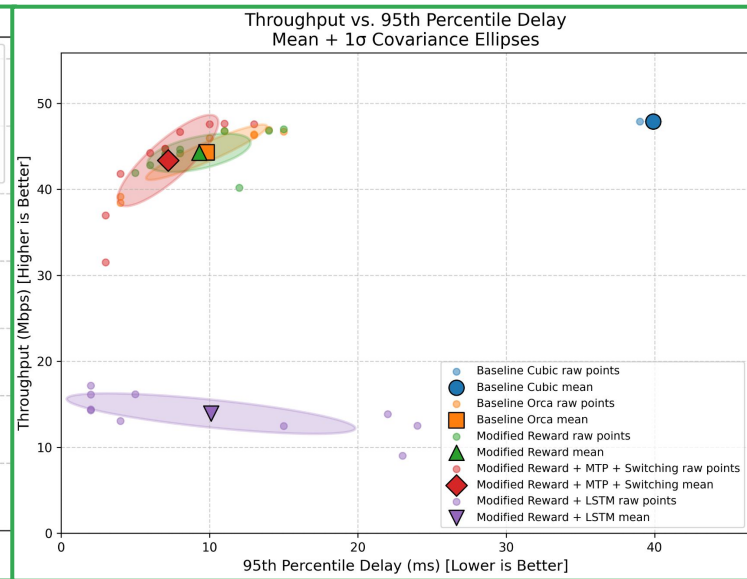
Step Trace.

Implementation Comparisons



Most modifications converge to the optimal traffic egress rate.

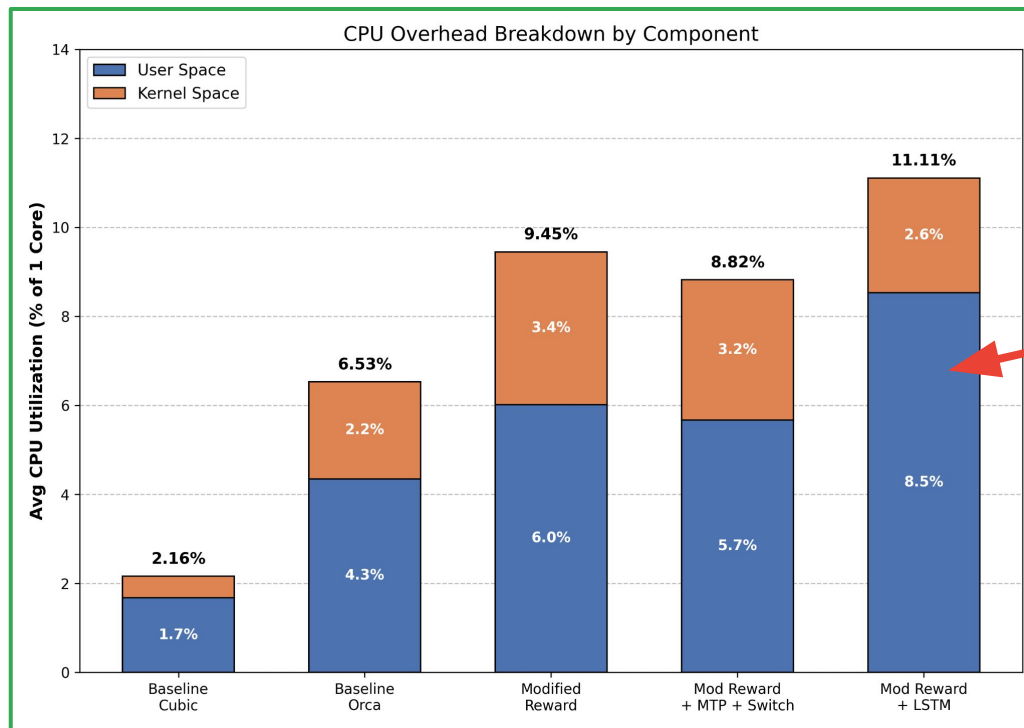
except for the LSTM :(



Cubic is stable in a steady-state network.

Wired48 Trace.

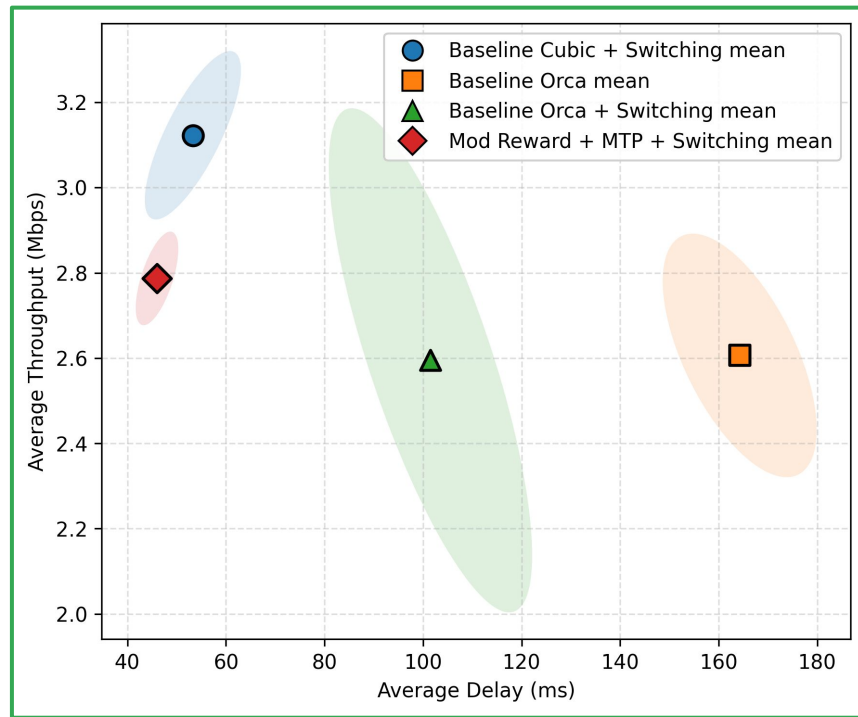
CPU Utilization



LSTM-DRL agent has super high userspace overhead.

Modifications → higher CPU usage.

Cellular Networks



The effect of algorithm switching on a cellular trace.

Improvements

Move inference to kernelspace to eliminate IPC latency via eBPF.

Have DRL agent predict congestion control algorithm rather than deterministic kernelspace state machine.

Create a three-stage model pipeline to determine MTP, fallback algorithm, and action instead of a single agent for all predictions.

Conclusion...

Is it worth switching to learning-based congestion control schemes?

Yes and no: DRL is fragile, but in many cases performs better than classical congestion control algorithms. When it fails, we need guardrails.

→ keep the reliability of Cubic/BBR/Westwood/etc. as a fallback, but let learned decisions drive when they can do better.

Questions?